Andy Garcia
Assignment 5
EECS 348
10/27

- Two GenAI programs used: ChatGPT and Google Gemini
- Accessed through browser
- Prompt:

Below is a working C++ Sudoku solver (single file, object-oriented, recursive depth-first search with backtracking). It reads puzzle1.txt through puzzle5.txt, which are 9x9 Sudoku puzzles with blanks shown as underscores.

Modify this code (do NOT rewrite it from scratch; keep the existing structure, function names, and comments, and only change or add what is needed) so that:

1. For each puzzle file puzzleN.txt, the program writes the results to a file named solutionN.txt (for example, puzzle3.txt -> solution3.txt).
2. If no solutions are found, solutionN.txt contains only the single line: No solution found.
3. If solutions are found, solutionN.txt contains ALL solutions, followed by a single final line: Solutions found: n (where n is the number of solutions).
4. Each solution is written in the same format as the input puzzle files.
5. All of the program's existing functionality must still work. Fix anything that is broken.
6. Add comments explaining what each added or changed line does, and mark every changed or added section with a comment saying it is a revision.

Return the complete modified C++ file.

Here is the code:

```
/*
Program name: Assignment4
Program description:
    a C++ program that solves sudoku puzzles
    using an object oriented, recursive approach
    with depth-first search and backtracking
Inputs: txt file containing an unsolved 9x9 sudoku puzzle (puzzle1.txt - puzzle5.txt)
Outputs:
    the puzzle filename, the unsolved puzzle,
    the solved puzzle if 1+ solutions exist,
    "no solution" if no solutions exist
Collaborators:
```

"What is meant with "const" at end of function declaration?"

(https://stackoverflow.com/questions/3141087/what-is-meant-with-const-at-end-of-function-declaration)
Other sources: Gemini
Author: Ana Gonzalez Yuil
Creation Date: 10.14.26
Revision Date: 10.14.26
Revisions: personal comments (indicated by ***), check for enough filled cells (at least 17)
*/

```
/**
 * @file Assignment4.cpp
 * @brief Object-based Sudoku Solver using Recursive Backtracking (DFS) in C++.
 *
 * This program reads partially solved Sudoku puzzles from text files where blank spaces
 * are represented by underscores ('_'). It uses a class-based approach, storing the
 * grid in nested vectors (lists), and solves each puzzle using depth-first search
 * with backtracking. It outputs the filename, the unsolved puzzle, all solutions
 * found, and the total solution count.
 */

#include <iostream>
#include <vector>
#include <string>
#include <fstream>

/**
 * @class SudokuPuzzle
 * @brief Represents a 9x9 Sudoku puzzle board and provides methods to load,
 *        validate, and solve the puzzle using recursive backtracking.
 */
class SudokuPuzzle {
private:
    // 2D vector (list of lists) to store the 9x9 grid characters ('1'-'9' or '_')
    std::vector<std::vector<char>> grid;

public:
    /**
     * @brief Construct a new Sudoku Puzzle object.
     * Initializes a 9x9 grid filled with placeholder underscores ('_').
     */
    SudokuPuzzle() {
        grid.resize(9, std::vector<char>(9, '_')); // *** changes size of grid after initialization
```

```
        // *** '9' sets the size of grid to 9.
        // *** 'std::vector<char>(9, '_')' initializes every element of grid to be a vector of size 9 with every element set to '_'
    }

    /**
     * @brief Loads a Sudoku puzzle from a text file.
     *
     * @param filename The path/name of the file to open.
     * @return true If the file was successfully opened and read.
     * @return false If the file could not be opened.
     */
    bool loadFromFile(const std::string& filename) {// *** takes input variable by reference (does not create a copy)
        int filledCells = 0; // *** holds amount of filled cells in unsolved sudoku grid

        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        // Read 9 rows and 9 columns from the file
        for (int i = 0; i < 9; ++i) { // ** ++i works the same as i++
            for (int j = 0; j < 9; ++j) {
                file >> grid[i][j];

                // *** implement check for at least 17 filled squares
                if (grid[i][j] != '_') {filledCells++;} // *** if the cell is not empty, add to filledCells count.
            }
        }
        file.close();

        if (filledCells < 17) {return false;} //if filledcells is less than 17, return false. (this prevents empty input)

        return true;
    }

    /**
     * @brief Prints the current state of the puzzle grid to standard output.
     */
    void print() const { // *** const keyword after function declaration makes it so that class function cannot change the input (StackOverflow)
        for (int i = 0; i < 9; ++i) {
```

```
        for (int j = 0; j < 9; ++j) {
            std::cout << grid[i][j] << " ";
        }
        std::cout << "\n";
    }
}

/**
 * @brief Validates if placing a specific character in a cell follows Sudoku rules.
 *
 * Checks if the value already exists in the given row, column, or 3x3 subgrid.
 *
 * @param row The row index (0-8) of the cell.
 * @param col The column index (0-8) of the cell.
 * @param val The character value ('1'-'9') to check.
 * @return true If the value is valid according to Sudoku rules.
 * @return false If the value violates row, column, or subgrid constraints.
 */
bool isValid(int row, int col, char val) const {
    // 1. Check the row and column constraints
    for (int i = 0; i < 9; ++i) {
        // *** checks every num in row to see if value already exists
        if (grid[row][i] == val) return false; // Found in row
        // *** checks every num in col to see if value already exists
        if (grid[i][col] == val) return false; // Found in column
    }

    // 2. Check the 3x3 subgrid constraint
    // *** startRow and startCol calculation:
    // *** this snaps the start of the loop to the constraints of the corresponding subgrid
    int startRow = row - (row % 3);
    int startCol = col - (col % 3);
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            if (grid[startRow + i][startCol + j] == val) {
                return false; // Found in the 3x3 subgrid
            }
        }
    }

    return val; // Valid placement
}

/**
```

```
 * @brief Iterates through the grid to find the next empty cell.
 *
 * @param row Reference variable updated with the row index of the empty cell.
 * @param col Reference variable updated with the column index of the empty cell.
 * @return true If an empty cell ('_') is found.
 * @return false If the entire grid is filled (no empty cells remain).
 */
bool findEmptyCell(int& row, int& col) const {
    for (row = 0; row < 9; ++row) {
        for (col = 0; col < 9; ++col) {
            if (grid[row][col] == '_') {
                return true; // Empty cell located
            }
        }
    }
    return false; // Puzzle is completely filled
}

/**
 * @brief Recursive Depth-First Search function with backtracking to find all solutions.
 *
 * Identifies an empty cell, iterates through candidate values ('1'-'9'),
 * places valid candidates, and recurses to the next empty cell. Backtracks
 * by resetting cells if a branch leads to a dead end.
 *
 * @param solutionCount Reference counter tracking the number of valid solutions found.
 * @param solutions Reference container storing all valid solution grids.
 */
void solveAll(int& solutionCount, std::vector<std::vector<std::vector<char>>>& solutions) {
    int row, col;

    // Base case: If no empty cell remains, a complete valid solution is achieved (k=81)
    if (!findEmptyCell(row, col)) {
        solutionCount++;
        solutions.push_back(grid);
        return;
    }

    // Try candidate values from '1' to '9' for the current empty cell
    for (char val = '1'; val <= '9'; ++val) {
        if (isValid(row, col, val)) {
            // Place candidate value into the cell
            grid[row][col] = val;
```

```
            // Move onto the next empty cell with a recursive function call
            solveAll(solutionCount, solutions);

            // Backtrack: Reset cell to empty if the recursive branch fails
            grid[row][col] = '_';
        }
    }
}

/**
 * @brief Helper function to process, solve, and print the results for a specific puzzle file.
 *
 * @param filename The file name containing the puzzle.
 */
static void processPuzzle(const std::string& filename) {
    std::cout << "========================================\n";
    std::cout << "Puzzle Filename: " << filename << "\n";
    std::cout << "========================================\n";

    SudokuPuzzle puzzle;

    // Attempt to load the puzzle from the file
    if (!puzzle.loadFromFile(filename)) {
        std::cout << "Error: Could not open file " << filename << " or not enough squares
filled.\n\n";
        return;
    }

    // Print the initial unsolved puzzle
    std::cout << "Unsolved Puzzle:\n";
    puzzle.print();
    std::cout << "\n";

    // Containers to store solutions and solution counts
    int solutionCount = 0;
    std::vector<std::vector<std::vector<char>>> solutions; //*** vector that holds each solved
sudoku grid

    // Execute backtracking solver
    puzzle.solveAll(solutionCount, solutions);

    // Print number of solutions
    std::cout << "Number of Solutions: " << solutionCount << "\n\n";
```

```
        // Print puzzle solution(s) or "no solution found" message
        if (solutionCount == 0) {
            std::cout << "no solution found\n\n";
        } else {
            // *** if solutions exist, print each solution using the size of the solutions vector as limit
            // *** s indicates which solution is being printed
            for (size_t s = 0; s < solutions.size(); ++s) {
                std::cout << "Solution " << (s + 1) << ":\n";
                for (int i = 0; i < 9; ++i) {
                    for (int j = 0; j < 9; ++j) {
                        std::cout << solutions[s][i][j] << " ";
                    }
                    std::cout << "\n";
                }
                std::cout << "\n";
            }
        }
    }
};

/**
 * @brief Main execution function.
 *
 * Iterates through the five required puzzle files and processes each one sequentially.
 */
int main() {
    // Array containing the names of the five target text files
    std::string puzzleFiles[] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // Process each puzzle file sequentially
    for (const std::string& filename : puzzleFiles) {
        SudokuPuzzle::processPuzzle(filename);
    }

    return 0;
}
```

ChatGPT’s Code:

```
/*
Program name: Assignment4
Program description:
   a C++ program that solves sudoku puzzles
   using an object oriented, recursive approach
   with depth-first search and backtracking
Inputs: txt file containing an unsolved 9x9 sudoku puzzle (puzzle1.txt - puzzle5.txt)
Outputs:
   the puzzle filename, the unsolved puzzle,
   the solved puzzle if 1+ solutions exist,
   "no solution" if no solutions exist
Collaborators:
   "What is meant with "const" at end of function declaration?"

(https://stackoverflow.com/questions/3141087/what-is-meant-with-const-at-end-of-function-declaration)
Other sources: Gemini
Author: Ana Gonzalez Yuil
Creation Date: 10.14.26
Revision Date: 10.27.26
Revisions: personal comments (indicated by ***), check for enough filled cells (at least 17)
REVISION: Added solution-file output for all five puzzles and corrected validation.
*/

/**
 * @file Assignment4.cpp
 * @brief Object-based Sudoku Solver using Recursive Backtracking (DFS) in C++.
 *
 * This program reads partially solved Sudoku puzzles from text files where blank spaces
 * are represented by underscores ('_'). It uses a class-based approach, storing the
 * grid in nested vectors (lists), and solves each puzzle using depth-first search
 * with backtracking. It outputs the filename, the unsolved puzzle, all solutions
 * found, and the total solution count.
 */

#include <iostream>
#include <vector>
#include <string>
#include <fstream>

// REVISION: Added this header to validate input characters using character checks.
```

```
#include <cctype>

/**
 * @class SudokuPuzzle
 * @brief Represents a 9x9 Sudoku puzzle board and provides methods to load,
 *        validate, and solve the puzzle using recursive backtracking.
 */
class SudokuPuzzle {
private:
    // 2D vector (list of lists) to store the 9x9 grid characters ('1'-'9' or '_')
    std::vector<std::vector<char>> grid;

public:
    /**
     * @brief Construct a new Sudoku Puzzle object.
     * Initializes a 9x9 grid filled with placeholder underscores ('_').
     */
    SudokuPuzzle() {
        grid.resize(9, std::vector<char>(9, '_')); // *** changes size of grid after initialization
        // *** '9' sets the size of grid to 9.
        // *** 'std::vector<char>(9, '_')' initializes every element of grid to be a vector of size 9 with
every element set to '_'
    }

    /**
     * @brief Loads a Sudoku puzzle from a text file.
     *
     * @param filename The path/name of the file to open.
     * @return true If the file was successfully opened and read.
     * @return false If the file could not be opened.
     */
    bool loadFromFile(const std::string& filename) { // *** takes input variable by reference (does
not create a copy)
        int filledCells = 0; // *** holds amount of filled cells in unsolved sudoku grid

        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        // Read 9 rows and 9 columns from the file
        for (int i = 0; i < 9; ++i) { // ** ++i works the same as i++
            for (int j = 0; j < 9; ++j) {
                // REVISION: Check that every expected cell can actually be read.
```

```
            if (!(file >> grid[i][j])) {
                file.close(); // REVISION: Close the input file before reporting malformed input.
                return false; // REVISION: Reject files with fewer than 81 cell values.
            }

            // REVISION: Reject anything except underscores and digits 1 through 9.
            if (grid[i][j] != '_' &&
                (grid[i][j] < '1' || grid[i][j] > '9')) {
                file.close(); // REVISION: Close the input file before returning.
                return false; // REVISION: Prevent invalid characters from entering the solver.
            }

            // *** implement check for at least 17 filled squares
            if (grid[i][j] != '_') {
                filledCells++; // *** if the cell is not empty, add to filledCells count.
            }
        }
    }

    file.close();

    // REVISION: Reject extra cell values so the input contains exactly one 9x9 grid.
    char extraCharacter;
    if (file >> extraCharacter) {
        return false;
    }

    // REVISION: Preserve the original minimum-filled-cell check.
    if (filledCells < 17) {
        return false; // This prevents empty or nearly empty input.
    }

    // REVISION: Verify the original clues do not already violate Sudoku rules.
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            if (grid[i][j] != '_') {
                char originalValue = grid[i][j]; // REVISION: Save the clue before temporarily
clearing it.
                grid[i][j] = '_'; // REVISION: Exclude the clue itself from the validation checks.
                bool valid = isValid(i, j, originalValue); // REVISION: Check the clue against its row,
column, and box.
                grid[i][j] = originalValue; // REVISION: Restore the original clue after checking it.

                if (!valid) {
```

```
                    return false; // REVISION: Reject puzzles with conflicting original clues.
                }
            }
        }
    }

    return true;
}

/**
 * @brief Prints the current state of the puzzle grid to standard output.
 */
void print() const { // *** const keyword after function declaration makes it so that class function cannot change the input (StackOverflow)
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            std::cout << grid[i][j] << " ";
        }
        std::cout << "\n";
    }
}

/**
 * @brief Validates if placing a specific character in a cell follows Sudoku rules.
 *
 * Checks if the value already exists in the given row, column, or 3x3 subgrid.
 *
 * @param row The row index (0-8) of the cell.
 * @param col The column index (0-8) of the cell.
 * @param val The character value ('1'-'9') to check.
 * @return true If the value is valid according to Sudoku rules.
 * @return false If the value violates row, column, or subgrid constraints.
 */
bool isValid(int row, int col, char val) const {
    // 1. Check the row and column constraints
    for (int i = 0; i < 9; ++i) {
        // *** checks every num in row to see if value already exists
        if (grid[row][i] == val) return false; // Found in row
        // *** checks every num in col to see if value already exists
        if (grid[i][col] == val) return false; // Found in column
    }

    // 2. Check the 3x3 subgrid constraint
    // *** startRow and startCol calculation:
```

```
    // *** this snaps the start of the loop to the constraints of the corresponding subgrid
    int startRow = row - (row % 3);
    int startCol = col - (col % 3);

    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            if (grid[startRow + i][startCol + j] == val) {
                return false; // Found in the 3x3 subgrid
            }
        }
    }

    // REVISION: Return true explicitly because a valid placement must evaluate to true.
    return true;
}

/**
 * @brief Iterates through the grid to find the next empty cell.
 *
 * @param row Reference variable updated with the row index of the empty cell.
 * @param col Reference variable updated with the column index of the empty cell.
 * @return true If an empty cell ('_') is found.
 * @return false If the entire grid is filled (no empty cells remain).
 */
bool findEmptyCell(int& row, int& col) const {
    for (row = 0; row < 9; ++row) {
        for (col = 0; col < 9; ++col) {
            if (grid[row][col] == '_') {
                return true; // Empty cell located
            }
        }
    }

    return false; // Puzzle is completely filled
}

/**
 * @brief Recursive Depth-First Search function with backtracking to find all solutions.
 *
 * Identifies an empty cell, iterates through candidate values ('1'-'9'),
 * places valid candidates, and recurses to the next empty cell. Backtracks
 * by resetting cells if a branch leads to a dead end.
 *
 * @param solutionCount Reference counter tracking the number of solutions found.
```

```
 * @param solutions Reference container storing all valid solution grids.
 */
void solveAll(int& solutionCount,
              std::vector<std::vector<std::vector<char>>>& solutions) {
    int row, col;

    // Base case: If no empty cell remains, a complete valid solution is achieved (k=81)
    if (!findEmptyCell(row, col)) {
        solutionCount++;
        solutions.push_back(grid);
        return;
    }

    // Try candidate values from '1' to '9' for the current empty cell
    for (char val = '1'; val <= '9'; ++val) {
        if (isValid(row, col, val)) {
            // Place candidate value into the cell
            grid[row][col] = val;

            // Move onto the next empty cell with a recursive function call
            solveAll(solutionCount, solutions);

            // Backtrack: Reset cell to empty if the recursive branch fails
            grid[row][col] = '_';
        }
    }
}

/**
 * @brief Helper function to process, solve, and print the results for a specific puzzle file.
 *
 * @param filename The file name containing the puzzle.
 */
static void processPuzzle(const std::string& filename) {
    std::cout << "========================================\n";
    std::cout << "Puzzle Filename: " << filename << "\n";
    std::cout << "========================================\n";

    // REVISION: Derive the output filename by replacing the "puzzle" prefix with "solution".
    // For example, "puzzle3.txt" becomes "solution3.txt".
    std::string outputFilename = "solution" + filename.substr(6);

    // REVISION: Open the output file so this puzzle's results are written separately.
    std::ofstream outputFile(outputFilename);
```

```
    // REVISION: Report an output-file error rather than silently losing the results.
    if (!outputFile.is_open()) {
        std::cout << "Error: Could not create output file " << outputFilename << ".\n\n";
        return;
    }

    SudokuPuzzle puzzle;

    // Attempt to load the puzzle from the file
    if (!puzzle.loadFromFile(filename)) {
        std::cout << "Error: Could not open file " << filename
                  << " or puzzle input is invalid.\n\n";

        // REVISION: Write only the required message when the puzzle cannot be loaded.
        outputFile << "No solution found.\n";
        outputFile.close(); // REVISION: Close the output file after writing the failure message.
        return;
    }

    // Print the initial unsolved puzzle
    std::cout << "Unsolved Puzzle:\n";
    puzzle.print();
    std::cout << "\n";

    // Containers to store solutions and solution counts
    int solutionCount = 0;
    std::vector<std::vector<std::vector<char>>> solutions; // *** vector that holds each solved
sudoku grid

    // Execute backtracking solver
    puzzle.solveAll(solutionCount, solutions);

    // Print number of solutions
    std::cout << "Number of Solutions: " << solutionCount << "\n\n";

    // Print puzzle solution(s) or "no solution found" message
    if (solutionCount == 0) {
        std::cout << "no solution found\n\n";

        // REVISION: The output file must contain this single line and nothing else.
        outputFile << "No solution found.\n";
    } else {
        // *** if solutions exist, print each solution using the size of the solutions vector as limit
```

```
        // *** s indicates which solution is being printed
        for (size_t s = 0; s < solutions.size(); ++s) {
            std::cout << "Solution " << (s + 1) << ":\n";

            // REVISION: Write each solution in the same space-separated format as the input.
            for (int i = 0; i < 9; ++i) {
                for (int j = 0; j < 9; ++j) {
                    std::cout << solutions[s][i][j] << " ";
                    outputFile << solutions[s][i][j] << " "; // REVISION: Save each cell to the solution
file.
                }
                std::cout << "\n";
                outputFile << "\n"; // REVISION: Preserve one row per line in the output file.
            }

            std::cout << "\n";
            outputFile << "\n"; // REVISION: Separate multiple solutions with a blank line.
        }

        // REVISION: Append the total solution count as the final line of the output file.
        outputFile << "Solutions found: " << solutionCount << "\n";
    }

    // REVISION: Explicitly close the output file after all results have been written.
    outputFile.close();
  }
};

/**
 * @brief Main execution function.
 *
 * Iterates through the five required puzzle files and processes each one sequentially.
 */
int main() {
  // Array containing the names of the five target text files
  std::string puzzleFiles[] = {
      "puzzle1.txt",
      "puzzle2.txt",
      "puzzle3.txt",
      "puzzle4.txt",
      "puzzle5.txt"
  };

  // Process each puzzle file sequentially
```

```
    for (const std::string& filename : puzzleFiles) {
        SudokuPuzzle::processPuzzle(filename);
    }

    return 0;
}
```

Gemini Code

```
/*
Program name: Assignment4
Program description:
   a C++ program that solves sudoku puzzles
   using an object oriented, recursive approach
   with depth-first search and backtracking
Inputs: txt file containing an unsolved 9x9 sudoku puzzle (puzzle1.txt - puzzle5.txt)
Outputs:
   the puzzle filename, the unsolved puzzle,
   the solved puzzle if 1+ solutions exist,
   "no solution" if no solutions exist
Collaborators:
   "What is meant with "const" at end of function declaration?"

(https://stackoverflow.com/questions/3141087/what-is-meant-with-const-at-end-of-function-declaration)
Other sources: Gemini
Author: Ana Gonzalez Yuil
Creation Date: 10.14.26
Revision Date: 10.27.26
Revisions: personal comments (indicated by ***), check for enough filled cells (at least 17), file output generation for solutions (Revision)
*/

/**
 * @file Assignment4.cpp
 * @brief Object-based Sudoku Solver using Recursive Backtracking (DFS) in C++.
 *
 * This program reads partially solved Sudoku puzzles from text files where blank spaces
 * are represented by underscores ('_'). It uses a class-based approach, storing the
 * grid in nested vectors (lists), and solves each puzzle using depth-first search
 * with backtracking. It outputs the filename, the unsolved puzzle, all solutions
 * found, and the total solution count to both console and solution text files.
 */

#include <iostream>
#include <vector>
#include <string>
#include <fstream>

/**
```

```
 * @class SudokuPuzzle
 * @brief Represents a 9x9 Sudoku puzzle board and provides methods to load,
 *        validate, and solve the puzzle using recursive backtracking.
 */
class SudokuPuzzle {
private:
    // 2D vector (list of lists) to store the 9x9 grid characters ('1'-'9' or '_')
    std::vector<std::vector<char>> grid;

public:
    /**
     * @brief Construct a new Sudoku Puzzle object.
     * Initializes a 9x9 grid filled with placeholder underscores ('_').
     */
    SudokuPuzzle() {
        grid.resize(9, std::vector<char>(9, '_')); // *** changes size of grid after initialization
        // *** '9' sets the size of grid to 9.
        // *** 'std::vector<char>(9, '_')' initializes every element of grid to be a vector of size 9 with
every element set to '_'
    }

    /**
     * @brief Loads a Sudoku puzzle from a text file.
     *
     * @param filename The path/name of the file to open.
     * @return true If the file was successfully opened and read.
     * @return false If the file could not be opened.
     */
    bool loadFromFile(const std::string& filename) {// *** takes input variable by reference (does
not create a copy)
        int filledCells = 0; // *** holds amount of filled cells in unsolved sudoku grid

        std::ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        // Read 9 rows and 9 columns from the file
        for (int i = 0; i < 9; ++i) { // ** ++i works the same as i++
            for (int j = 0; j < 9; ++j) {
                file >> grid[i][j];

                // *** implement check for at least 17 filled squares
                if (grid[i][j] != '_') {filledCells++;} // *** if the cell is not empty, add to filledCells count.
```

```
            }
        }
        file.close();

        if (filledCells < 17) {return false;} //if filledcells is less than 17, return false. (this prevents empty input)

        return true;
    }

    /**
     * @brief Prints the current state of the puzzle grid to standard output.
     */
    void print() const { // *** const keyword after function declaration makes it so that class function cannot change the input (StackOverflow)
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                std::cout << grid[i][j] << " ";
            }
            std::cout << "\n";
        }
    }

    /**
     * @brief Validates if placing a specific character in a cell follows Sudoku rules.
     *
     * Checks if the value already exists in the given row, column, or 3x3 subgrid.
     *
     * @param row The row index (0-8) of the cell.
     * @param col The column index (0-8) of the cell.
     * @param val The character value ('1'-'9') to check.
     * @return true If the value is valid according to Sudoku rules.
     * @return false If the value violates row, column, or subgrid constraints.
     */
    bool isValid(int row, int col, char val) const {
        // 1. Check the row and column constraints
        for (int i = 0; i < 9; ++i) {
            // *** checks every num in row to see if value already exists
            if (grid[row][i] == val) return false; // Found in row
            // *** checks every num in col to see if value already exists
            if (grid[i][col] == val) return false; // Found in column
        }

        // 2. Check the 3x3 subgrid constraint
```

```
    // *** startRow and startCol calculation:
    // *** this snaps the start of the loop to the constraints of the corresponding subgrid
    int startRow = row - (row % 3);
    int startCol = col - (col % 3);
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            if (grid[startRow + i][startCol + j] == val) {
                return false; // Found in the 3x3 subgrid
            }
        }
    }

    return true; // Revision: fixed return value from char 'val' to explicit boolean true
}

/**
 * @brief Iterates through the grid to find the next empty cell.
 *
 * @param row Reference variable updated with the row index of the empty cell.
 * @param col Reference variable updated with the column index of the empty cell.
 * @return true If an empty cell ('_') is found.
 * @return false If the entire grid is filled (no empty cells remain).
 */
bool findEmptyCell(int& row, int& col) const {
    for (row = 0; row < 9; ++row) {
        for (col = 0; col < 9; ++col) {
            if (grid[row][col] == '_') {
                return true; // Empty cell located
            }
        }
    }
    return false; // Puzzle is completely filled
}

/**
 * @brief Recursive Depth-First Search function with backtracking to find all solutions.
 *
 * Identifies an empty cell, iterates through candidate values ('1'-'9'),
 * places valid candidates, and recurses to the next empty cell. Backtracks
 * by resetting cells if a branch leads to a dead end.
 *
 * @param solutionCount Reference counter tracking the number of valid solutions found.
 * @param solutions Reference container storing all valid solution grids.
 */
```

```
    void solveAll(int& solutionCount, std::vector<std::vector<std::vector<char>>>& solutions) {
        int row, col;

        // Base case: If no empty cell remains, a complete valid solution is achieved (k=81)
        if (!findEmptyCell(row, col)) {
            solutionCount++;
            solutions.push_back(grid);
            return;
        }

        // Try candidate values from '1' to '9' for the current empty cell
        for (char val = '1'; val <= '9'; ++val) {
            if (isValid(row, col, val)) {
                // Place candidate value into the cell
                grid[row][col] = val;

                // Move onto the next empty cell with a recursive function call
                solveAll(solutionCount, solutions);

                // Backtrack: Reset cell to empty if the recursive branch fails
                grid[row][col] = '_';
            }
        }
    }

    /**
     * @brief Helper function to process, solve, and print the results for a specific puzzle file.
     *
     * @param filename The file name containing the puzzle.
     */
    static void processPuzzle(const std::string& filename) {
        std::cout << "========================================\n";
        std::cout << "Puzzle Filename: " << filename << "\n";
        std::cout << "========================================\n";

        SudokuPuzzle puzzle;

        // Attempt to load the puzzle from the file
        if (!puzzle.loadFromFile(filename)) {
            std::cout << "Error: Could not open file " << filename << " or not enough squares
filled.\n\n";
            return;
        }
```

```
        // Print the initial unsolved puzzle
        std::cout << "Unsolved Puzzle:\n";
        puzzle.print();
        std::cout << "\n";

        // Containers to store solutions and solution counts
        int solutionCount = 0;
        std::vector<std::vector<std::vector<char>>> solutions; //*** vector that holds each solved
sudoku grid

        // Execute backtracking solver
        puzzle.solveAll(solutionCount, solutions);

        // Print number of solutions
        std::cout << "Number of Solutions: " << solutionCount << "\n\n";

        // Print puzzle solution(s) or "no solution found" message
        if (solutionCount == 0) {
            std::cout << "no solution found\n\n";
        } else {
            // *** if solutions exist, print each solution using the size of the solutions vector as limit
            // *** s indicates which solution is being printed
            for (size_t s = 0; s < solutions.size(); ++s) {
                std::cout << "Solution " << (s + 1) << ":\n";
                for (int i = 0; i < 9; ++i) {
                    for (int j = 0; j < 9; ++j) {
                        std::cout << solutions[s][i][j] << " ";
                    }
                    std::cout << "\n";
                }
                std::cout << "\n";
            }
        }

        // Revision: generate corresponding solution filename by replacing "puzzle" with "solution"
        std::string outFilename = filename;
        size_t pos = outFilename.find("puzzle");
        if (pos != std::string::npos) {
            outFilename.replace(pos, 6, "solution");
        }

        // Revision: open the output file stream to write results
        std::ofstream outFile(outFilename);
        if (outFile.is_open()) {
```

```
        // Revision: if no solutions are found, write exactly "No solution found." to file
        if (solutionCount == 0) {
            outFile << "No solution found.\n";
        } else {
            // Revision: write all solutions in puzzle format followed by the total solution count
            for (size_t s = 0; s < solutions.size(); ++s) {
                for (int i = 0; i < 9; ++i) {
                    for (int j = 0; j < 9; ++j) {
                        outFile << solutions[s][i][j] << (j == 8 ? "" : " ");
                    }
                    outFile << "\n";
                }
            }
            outFile << "Solutions found: " << solutionCount << "\n";
        }
        // Revision: close the output file stream
        outFile.close();
    }
  }
};

/**
 * @brief Main execution function.
 *
 * Iterates through the five required puzzle files and processes each one sequentially.
 */
int main() {
    // Array containing the names of the five target text files
    std::string puzzleFiles[] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // Process each puzzle file sequentially
    for (const std::string& filename : puzzleFiles) {
        SudokuPuzzle::processPuzzle(filename);
    }

    return 0;
}
```

Code Analysis:

Comparison:

Correctness:

Gemini: Fixed return bug on isValid() where the original code returned val which returned the ASCII value which does implicitly return true however it was corrected to explicitly return true for clarity.

ChatGPT: Also did the same fix of isValid() and writes the No solution found. When the puzzle cannot be loaded.

Execution Time:

Neither AI improved on the execution time. The worst case still remained O(9^n) where n is the amount of empty cells in the puzzle.

Space Complexity:

Both programs have O(n) recursion depth and both kept each solution in its memory until the end of the program making it O(81 x S) for S solutions.

Maintainability:

Gemini: Added a piece of code at the end of processPuzzle that builds the solutionN.txt file. Marked each change with a “Revision:” comment. It does not print an error message if the file cannot be created.

ChatGPT: Remade the loadFromFile function to check that each value actually reads. It also called the file.close() function manually making it harder to test because it made this function so large. Improved output files by separating them to improve readability.

Choice of base for changes: ChatGPT
I chose ChatGPT’s changes as the base because it was able to reject an incorrect puzzle when given to it. It also rejected short files and incorrect characters. The two both had equal speed and memory so that did not affect the decision.

Code Changes: (assisted by Claude)

1. Improved correctness by deleting an early file.close() that prevented a further value check from occurring within the loadFromFile class.

2. Improved maintainability by moving ChatGPT's generated conflict check from loadFromFile into a new givensAreConsistent() function and calling it inside loadFromFile. Also removed an unused inclusion of a cctype library that ChatGPT had placed in the file and never used.